Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Departamentul Ingineria Software şi Automatică

Report on Laboratory Work No. 8

Process Calls and Pipes Inside xv6

Operating Systems

Done by: st. gr. FAF-241, Caţ Chiril

Checked by: asist. univ. Reiţman Patricia

Chişinău – 2026

Contents

1 Aim, deliverables, and build **1**

2 Part 1: a program starts another program **2**

2.1 How the process moves through the three calls 2

2.2 Changing the program to ls 3

2.3 A failed exec and its exit status 3

2.4 Comparison with the Lab 7 calls 4

3 Part 2: the pingpong exchange **4**

3.1 Why there are two pipes 4

3.2 What orders ping before pong 5

4 Handout questions and answers **6**

4.1 Part 1: run inside xv6 6

4.2 Part 2: pingpong 6

5 Verification and conclusion **7**

1 Aim, deliverables, and build

This lab repeats the process experiment from Lab 7 inside xv6 and then uses two pipes to exchange a byte between a parent and a child. The aim is to connect the shell-level behavior observed in Lab 7 to the smaller xv6 system-call interface: `fork` creates a child, `exec` replaces its program, `wait` reaps it, and `read`/`write` move bytes through pipes.

I added `user/run.c` and `user/pingpong.c` to the integrated `os/xv6-riscv` tree. Both names were added to the Makefile's `UPROGS` list, so `mkfs` includes their binaries in `fs.img`. Matching source files and the Makefile are in `lab8/code/`; this report's source is in `lab8/report_files/`. I used the local RISC-V compiler and QEMU on 26 October 2026. The relevant build and launch commands were:

```
$ make clean
$ make fs.img
... user/run.c ... user/pingpong.c ...
nmeta 47 (boot, super, log blocks 31, inode blocks 13,
          bitmap blocks 1) blocks 1953 total 2000
$ make qemu
xv6 kernel is booting
init: starting sh
$
```

The compiler completed with xv6's `-Wall -Werror` settings. For the requested `ls` and nonexistent-program experiments, I temporarily changed `user/run.c`, rebuilt the user binary and disk image, and booted QEMU again. I restored the submitted `echo` version and forced a final rebuild of both `run` and `fs.img`. A final boot verified the restored image.

The artifacts serve different purposes. The two C files are the actual xv6 user programs. The Makefile is needed to place their binaries in the image; copying a C file into `user/` alone would not make it available at the xv6 shell. The `notes.md` file preserves the short Observe answers, while this report explains the call sequence and the evidence in more detail. Each QEMU run began from the shell prompt shown in the transcripts. The PIDs are from those particular boots and are observations, not fixed values required by the programs.

2 Part 1: a program starts another program

2.1 How the process moves through the three calls

The final `run.c` first prints the parent's PID. The return value of `fork()` distinguishes the two paths: zero in the child and the child's PID in the parent. The child builds an argument vector ending in a null pointer and executes `echo`. The parent calls `wait(&status)` and prints the status after the child has ended. The core of the implementation is:

```
int pid = fork();
if(pid == 0){
  char *args[] = { "echo", "hello", "from", "the", "child", 0 };
  exec("echo", args);
  printf("exec failed\n");
  exit(1);
}
int status;
wait(&status);
printf("parent: child %d ended with status %d\n", pid, status);
```

Successful `exec` replaces the child process's address space. It does not create a second child and does not return to the error print in `run.c`. The child keeps its PID while the new `echo` program writes the supplied words. The parent remains blocked in `wait` until that program exits.

```
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
```

The observed PID 4 is the child created by PID 3. The words `hello from the child` come from `echo`, not from a print statement in `run.c`. The child's successful exit produced status 0, which the parent reported after `wait` returned.

2.2 Changing the program to ls

I replaced only the child's argument vector and `exec` call with `char *args[] = { "ls", 0 };` and `exec("ls", args);`. The next boot ran `ls` in the same child process. Its output included the new lab programs, confirming they were present in the rebuilt file system image. The complete directory listing from that run was:

```
$ run
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
sleep          2 3 35824
args           2 4 35824
mystat         2 5 37008
find           2 6 39064
run            2 7 36496
pingpong       2 8 37672
cat            2 9 37344
echo           2 10 36160
forktest       2 11 17864
grep           2 12 40856
init           2 13 36504
kill           2 14 36064
ln             2 15 35864
ls             2 16 39400
mkdir          2 17 36120
rm             2 18 36104
sh             2 19 59680
stressfs       2 20 36944
usertests      2 21 204344
grind          2 22 51976
wc             2 23 38344
zombie         2 24 35400
logstress      2 25 38112
forphan        2 26 36864
dorphan        2 27 36320
sync           2 28 35496
console        3 29 0
parent: child 4 ended with status 0
```

2.3 A failed exec and its exit status

For the third run, the child attempted `exec("nosuchprog", args)`. There was no matching executable in the image, so `exec` returned to `run.c`. The error print and `exit(1)` then executed:

```
$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
```

The status is 1 because xv6's example explicitly calls `exit(1)` after failed `exec`. Lab 7's POSIX version calls `exit(127)` after failed `execlp`, so its parent reports 127. The different values are choices made by the example programs; `exec` itself does not assign them.

2.4 Comparison with the Lab 7 calls

Step	Lab 7 POSIX program	Lab 8 xv6 program
Create child	`fork()`	`fork()`
Replace child	`execlp("ls", "ls", "-l", ...)`	`exec("echo", args)`
Collect result	`waitpid(pid, &status, 0)` and decode it	`wait(&status)`

The same three stages occur in both environments. xv6 offers a smaller interface: the executable path and argument array are passed to `exec`, and `wait` returns the child's PID while writing its exit status through the pointer. The Lab 7 program used the POSIX library call `execlp` and `waitpid`, with POSIX status decoding. The parent and child remain separate processes in both runs; only the child's program is replaced.

3 Part 2: the pingpong exchange

3.1 Why there are two pipes

`pingpong.c` creates both pipes before `fork`, so the parent and child inherit descriptors for both. A pipe has a read end and a write end and carries bytes from the latter to the former. One pipe carries the request from parent to child; the other carries the reply in the reverse direction. A single pipe would not give the two messages separate routes: both processes could compete to read the same byte, including a process reading its own write.

Process	Descriptors it keeps	Descriptors it closes
Parent	`p2c[1]` to send; `c2p[0]` to receive	`p2c[0]`, `c2p[1]`
Child	`p2c[0]` to receive; `c2p[1]` to send	`p2c[1]`, `c2p[0]`

Each process closes its retained ends after use as well. This avoids unnecessary open descriptors and lets readers observe end-of-file if their peer exits without sending data. The program checks the return values of `pipe`, `fork`, `read`, and `write` and reports a short error message if one fails.

3.2 What orders ping before pong

The parent begins with a byte `'x'` and writes exactly one byte to `p2c[1]`. The child reads it from `p2c[0]`, prints `received ping`, then writes that same byte to `c2p[1]`. The parent calls `read(c2p[0], &b, 1)` before printing `received pong`. If the child has not replied, this `read` blocks. The order follows from the byte exchange; there is no timer or separate sleep call.

```
// Parent: send request, then wait for the reply byte.
write(p2c[1], &b, 1);
close(p2c[1]);
read(c2p[0], &b, 1);
printf("%d: received pong\n", getpid());
wait(&status);

// Child: receive request, then return the same byte.
read(p2c[0], &b, 1);
printf("%d: received ping\n", getpid());
write(c2p[1], &b, 1);
```

These excerpts show the successful path; the submitted C source also checks each call's result and closes descriptors on error. The `wait(&status)` after the parent's print reaps the child. It does not make the parent wait for the reply: that happens at `read(c2p[0], &b, 1)`.

```
$ pingpong
6: received ping
5: received pong
$ pingpong
8: received ping
7: received pong
```

The PIDs changed between runs because each invocation created a new parent and child. In both runs, the child printed first and the parent printed after reading the reply. The byte itself is not displayed; the messages identify which side has completed its read.

4 Handout questions and answers

These are the required Observe questions from the Lab 8 handout, followed by the answers supported by the runs and source code above.

4.1 Part 1: run inside xv6

1. **Question:** Write the three calls that `run.c` uses in xv6 and the three calls `run.c` used on Linux in Lab 7. What is the same?

   Answer: xv6 uses `fork()`, `exec("echo", args)`, and `wait(&status)`. The Lab 7 POSIX version uses `fork()`, `execlp(...)`, and `waitpid(pid, &status, 0)`. Both programs create a child, replace its program, and have the parent collect its exit status.

2. **Question:** Who prints "hello from the child": the code of `run`, or `echo`?

   Answer: `echo` prints it. `run` puts those words in the argument vector; after successful `exec`, the child's `run` code has been replaced by `echo`.

3. **Question:** Why is the status 1 here and 127 on Linux? (Look at the exit line after `exec` in both programs.)

   Answer: The xv6 example calls `exit(1)` after failed `exec`; the Lab 7 POSIX example calls `exit(127)` after failed `execlp`. The parent reports the code chosen by its child.

4.2 Part 2: pingpong

1. **Question:** Why do you need TWO pipes, and not one for both directions?

   Answer: A pipe has one read end and one write end and carries bytes in one direction. `p2c` sends the request from parent to child; `c2p` sends the reply from child to parent. Separate streams prevent the two sides from competing to read the same byte.

2. **Question:** Nothing in your code says "wait until the child has replied". Which call makes the parent wait?

   Answer: The parent's `read(c2p[0], &b, 1)` blocks until the child writes the reply byte (or closes the pipe). The later `wait(&status)` only reaps the child.

5 Verification and conclusion

I tested the final `run` and `pingpong` programs after restoring the `echo` source and forcing a fresh image build. The final QEMU run again printed `hello from the child`, a status of 0, and two ping-before-pong pairs. The source copies in `lab8/code/` matched the corresponding xv6 files byte for byte.

The experiments show that xv6 implements the same basic process lifecycle as the POSIX example: a child is created, replaced with a new program, and reaped. A failed `exec` returns to the child and allows it to choose an error status. Two directed pipes and a blocking read provide the request-and-reply ordering in `pingpong`.